COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	August 20, 2026	**DATE SUBMITTED:**	August 27, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6.1 SEARCHING TECHNIQUES

1. PROCEDURE OUTPUT

ILO A: Create C++ code for searching data using different techniques

We will implement our searching technique on data filled with random values. Then, we will define the capacity of data elements in our dataset. This will be used to define your dataset. Once you are sure that you can generate random integers for your dataset, you can delete the second iterative loop and move on with the creation of the different searching techniques.

This implementation will also require the use of linked lists, so create a file “nodes.h” in your C++ project which will contain the definition of our single linked list node. Followed by a function that will provide us with the address of an object that points to null. Once this has been made, you are free to proceed to the next parts.

Verify this by including a screenshot of the generated data upon successful implementation of the code in the table. **(Table 6-1. Data Generated and Observations)**

Screenshot	**My Code: (main.cpp)** main.cpp > ... 1 #include <iostream> 2 #include <cstdlib> // For generating random integers 3 #include <time.h> // Will be used for our seeding function 4 5 const int max_size = 50; 6 7 int main() { 8 9 // Generate random dataset 10 int dataset[max_size]; 11 srand(time(0));

```
12
13      for (int i = 0; i < max_size; i++) {
14          dataset[i] = rand() % 100;
15      }
16
17      for (int i = 0; i < max_size; i++) {
18          std::cout << dataset[i] << " ";
19      }
20  }
```

My Code: (node.h)

node.h > ...

```
1   #ifndef NODE_H
2   #define NODE_H
3
4   template <typename T>
5   class Node{
6   public:
7       T data;
8       Node *next;
9   };
10
11  template <typename T>
12  Node<T> *new_node(T newData){
13      Node<T> *newNode = new Node<T>();
14      newNode->data = newData;
15      newNode->next = nullptr;
16      return newNode;
17  }
18
19  #endif
```

Observations	In the ***main.cpp***, the ***const int max_size = 50*** was used to set the capacity of the ***dataset array*** to 50 elements, which means it can store 50 integer values. The ***srand(time(0))*** was also used to seed the random number generator based on the current time, so the generated values can be different whenever the program is run. The first ***for*** loop, using ***dataset[i] = rand() % 100***, was important because it fills all 50 positions of the array with random integers from 0 to 99. The second ***for*** loop was then used to display the values using ***std::cout***, making it possible to check if the random data was generated correctly. For the ***node.h***, the ***Node*** template class was created to provide the structure for a singly linked list. The ***T data*** stores the value of the node, while the ***Node *next*** stores the address of the next node. The ***new_node()*** function was also important because it creates a new node using ***new Node<T>()***, places the given value into ***newNode->data***, and sets ***newNode->next = nullptr,*** which means the node does not point to another node yet. After this, the function returns the address of the created node.

Table 6-1. Data Generated and Observations

A.1 Sequential Search

Create a separate file implementation called “searchingAlgo.h” which we will use for the implementation of the algorithms.

Create a function called linearSearch that accepts necessary parameters to execute the algorithm described above. Indicate your code, output, and observations in the table.
(Table 6-2a. Linear Search for Arrays)

Code	**My Code: (searchingAlgo.h)** searchingAlgo.h > linearSearchLL<T>(Node<T>*, T) 1 #ifndef SEARCHINGALGO_H 2 #define SEARCHINGALGO_H 3 4 #include <iostream> 5 6 template <typename T> 7 void linearSearchArr(int arrSize, T dataFind, T list[]) { 8 9 // Step 1: I := 0 10 int i = 0; 11

```
    // Repeat while I <= n
    while (i < arrSize) {

        // If (item = data[i])
        if (dataFind == list[i]) {

            // Print "Searching is successful"
            std::cout << "Searching is successful. \n";
            return;
        }

        i++;
    }

    // Print "Searching is Unsuccessful"
    std::cout << "Searching is unsuccessful." << std::endl;
}
```

My Code: (main.cpp)

main.cpp > main()

```
#include <iostream>
#include <cstdlib> // For generating random integers
#include <time.h> // Will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    // Generate random dataset
    int dataset[max_size];
    srand(time(0));

    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand() % 100;
    }

    for (int i = 0; i < max_size; i++) {
        std::cout << dataset[i] << " ";
    }

    std::cout << std::endl;
```

```
24
25      // Create linear search for arrays
26      int userInput;
27      std::cout << "\nLinear Search for Arrays:" << std::endl;
28      std::cout << "---------------------------------" << std::endl;
29      std::cout << "What number to search? ";
30      std::cin >> userInput;
31      linearSearchArr(max_size, userInput, dataset);
32      std::cout << std::endl;
33
```

Output

```
6 38 19 76 21 26 71 34 11 36 47 77 87
 73 13 13 67 65 78 30 2 84 12 97 53 5
1 77 43 51 76 62 2 30 25 69 62 36 95
0 62 2 6 19 63 64 22 23 4 21 70

Linear Search for Arrays:
---------------------------------
What number to search? 77
Searching is successful.
```

Observations

The sequential search was successfully implemented using the ***linearSearchArr()*** function in ***searchingAlgo.h***. The ***arrSize***, ***dataFind***, and ***list[]*** parameters are important because they provide the array size, searched value, and dataset. The ***int i = 0*** starts the search at the first element, while the ***while (i < arrSize)*** loop checks each value one by one. The ***if (dataFind == list[i])*** compares the searched value with the current element. If a match is found, ***"Searching is successful."*** is displayed and the ***return*** stops the search. If no match is found, ***"Searching is unsuccessful."*** is displayed. In the main file, ***userInput*** gets the value to search, while ***linearSearchArr(max_size, userInput, dataset)*** sends the needed data to the function. Finally, the code worked correctly and showed how sequential search checks each element in order until a match is found or the array ends.

Table 6-2a. Linear Search for Arrays

Create a linked list that has your first name per node's data. Then, link each node to each other. Create a sequential search algorithm for linked lists by modifying the earlier function called linearLS. This function must take the head node of your linked list and a variable dataFind that it must search for within the given list.

The sequential search algorithm does not change even if it operates on a linked list. Show your modified code from earlier so that it will operate on a linked list. Screenshot your code and output with corresponding observations/notes in the table.

(Table 6-2b. Linear Search for Linked List)

Screenshot	**My Code: (searchingAlgo.h)**
	
	`template <typename T>`
	`void linearSearchLL(Node<T>* head, T dataFind) {`
	
	`    while (head != nullptr) {`
	
	`        if (head->data == dataFind) {`
	`            std::cout << "Searching is successful.\n";`
	`            return;`
	`        }`
	
	`        head = head->next;`
	`    }`
	
	`    std::cout << "Searching is unsuccessful." << std::endl;`
	`}`
	
	My Code: (main.cpp)
	
	`    // Create linear search for linked list`
	`    Node<char>* name1 = new_node('C');`
	`    Node<char>* name2 = new_node('A');`
	`    Node<char>* name3 = new_node('R');`
	`    Node<char>* name4 = new_node('L');`
	`    Node<char>* name5 = new_node('O');`
	
	`    name1->next = name2;`
	`    name2->next = name3;`
	`    name3->next = name4;`

```
44      name4->next = name5;
45      name5->next = nullptr;
46
47      std::cout << "Linear Search for Linked List:" << std::endl;
48      std::cout << "---------------------------------" << std::endl;
49      linearSearchLL(name1, 'R');
50      linearSearchLL(name2, 'r');
51      std::cout << std::endl;
52
```

Output

```
Linear Search for Linked List:
---------------------------------
Searching is successful.
Searching is unsuccessful.
```

Observations

The sequential search for the linked list was successfully implemented using ***linearSearchLL()***. The important part of the modified code is ***linearSearchLL(Node<T>* head, T dataFind)***, where ***head*** is used to start from the first node and ***dataFind*** is the value being searched. Instead of using ***list[i]*** like in the array search, the code uses ***head->data*** to check the value of each node and ***head = head->next*** to move to the next node. The ***while (head != nullptr)*** loop continues the search until the end of the linked list is reached. In the main file, the nodes containing the letters ***C, A, R, L,*** and ***O*** were created and connected using ***name1->next, name2->next***, and the other next pointers. The ***linearSearchLL(name1, 'R')*** call ***successfully*** found the letter ***R***, while ***linearSearchLL(name2, 'r')*** was ***unsuccessful*** because lowercase ***r*** is different from uppercase ***R***. Finally, the modified code worked correctly and showed that sequential search can still check each value one by one even when the data is stored in a linked list.

Table 6-2b. Linear Search for Linked List

B.1 Binary Search

This algorithm must also be included in your “searchingAlgo.h” file.

Create a function called binarySearch that accepts necessary parameters to execute the algorithm described above. Indicate your code, output, and observations in the table.

(Table 6-3a. Binary Search for Arrays)

Screenshot	**My Code: (searchingAlgo.h)**
	
	`template <typename T>`
	`void binarySearchArr(int arrSize, T dataFind, T list[]) {`
	
	`    // Define low, high, and middle`
	`    int low = 0, up = arrSize - 1, middle;`
	
	`    // Loop while low is less than or equal to high`
	`    while (low <= up) {`
	
	`        // Compute for middle`
	`        middle = (int)((low + up) / 2);`
	
	`        // If data found is middle -> data found -> return`
	`        if (dataFind == list[middle]) {`
	`            std::cout << "Searching is successful.\n";`
	`            return;`
	
	`        }`
	
	`        // If data find < data middle -> new high`
	`        else if (dataFind < list[middle]) {`
	`            up = middle - 1;`
	`        }`
	
	`        else {`
	
	`            // Else update low = middle + 1`
	`            low = middle + 1;`
	`        }`
	`    }`

```
75
76      // Search is not found
77      std::cout << "Searching is unsuccessful.\n";
78  }
```

My Code: (main.cpp)

```
53      // Create binary search for arrays
54      std::cout << "Binary Search for Arrays:" << std::endl;
55      std::cout << "---------------------------------" << std::endl;
56      int numbers[] = {15, 20, 25, 35, 40, 45, 55, 60, 65, 75};
57      binarySearchArr(10, 45, numbers);
58      binarySearchArr(10, 50, numbers);
59      std::cout << std::endl;
60
```

Output

```
Binary Search for Arrays:
---------------------------------
Searching is successful.
Searching is unsuccessful.
```

Observations

The binary search for arrays was successfully implemented using the ***binarySearchArr()*** function in the ***searchingAlgo.h*** file. The important code ***int low = 0, up = arrSize - 1, middle*** was used to set the starting, ending, and middle positions for the search. The ***while (low <= up)*** loop continues the search while there is still a valid range, while ***middle = (int)((low + up) / 2)*** finds the middle element. The ***if (dataFind == list[middle])*** checks if the searched value is found, while the ***else if*** and ***else*** statements move the search to the left or right half. In the main file, the ***numbers*** array was arranged in ascending order, which is important for binary search to work correctly. The ***binarySearchArr(10, 45, numbers)*** call ***successfully*** found ***45***, while ***binarySearchArr(10, 50, numbers)*** showed an ***unsuccessful*** search because ***50*** is not in the array. Finally, the code worked correctly and showed how binary search can reduce the search area by checking the middle and removing the half that does not need to be searched.

Table 6-3a. Binary Search for Arrays

Next, just like in the linked list with linear search, we will perform binary search in a linked list. However, there are pre-requisites for this to work: the linked list must have ordered values and we must know how many nodes exist in your list. An optional snippet of code can be added here to verify that our data is actually stored onto a linked list as intended.

Binary search still works the same, however. We need to create a function that will enable us to find the middle node. This function, getMiddle, must take the first and last node as its parameters. It follows the algorithm described below:

1. Traverse the singly linked list using two pointers.
2. Move one pointer by one step ahead and the other pointer by two steps.
3. When the fast pointer reaches the end of the singly linked list, the slow pointer will reach the middle of the singly linked list.
4. Return slow pointer address.

We will use this function to find the middle node of our linked list to properly search through the sorted list. Next, another function to implement the binary search. Step 2 of this algorithm uses the function from the algorithm above.

1. The start node is set to the head of the list and the last node is set to NULL.
2. Middle element is calculated using the two pointers approach discussed above.
3. If the middle element is the same as the key to be searched, we return it.
4. Else if the middle element is greater than the key to be searched, we have to search is the right side of the singly linked list. So, we set the start pointer to the next of the middle elements.
5. Else if the middle element is less than the key to be searched, we have to search is the left side of the singly linked list. So, we set the last pointer to the middle element.If the key is found or the entire linked list gets traversed, we break the loop.

Create the C++ code for this binary search algorithm and provide the output in the three cases shown in the table. Screenshot and observation are required in the table.

(Table 6-3b. Binary Search for Linked List)

Screenshot	**My Code: (searchingAlgo.h)** 80 template <typename T> 81 Node<T>* getMiddle(Node<T>* start, Node<T>* last) { 82

```
83      Node<T>* slow = start;
84      Node<T>* fast = start;
85
86      while (fast != last && fast->next != last) {
87
88          // Move slow one step
89          slow = slow->next;
90
91          // Move fast two steps
92          fast = fast->next->next;
93      }
94
95      return slow;
96  }
```

```
98  template <typename T>
99  void binarySearchLL(Node<T>* start, T dataFind) {
100
101     // Step 1: Set start to the head of the list, set last to null
102     Node<T>* last = nullptr;
103
104     // Continue searching while the search range is valid (not yet last)
105     while (start != last) {
106
107         // Step 2: Find the middle node using getMiddle()
108         Node<T>* middle = getMiddle(start, last);
109
110         // Step 3: If middle element is equal to the key -> search is successful
111         if (middle->data == dataFind) {
112             std::cout << "Searching is successful.\n";
113             return;
114         }
115
116         // Step 4: If the middle element is greater than the key -> search the left side
117         else if (middle->data > dataFind) {
118             last = middle;
119         }
120
```

```
121         // Step 5: If the middle element is less than the key, search the right side
122         else {
123             start = middle->next;
124         }
125     }
126
127     // Step 6: If the search range was traversed without finding the key
128     std::cout << "Searching is unsuccessful.\n";
129 }
130
131 #endif
```

My Code: (main.cpp)

```
61      // Create binary search for linked list
62      Node<int>* value1 = new_node(12);
63      Node<int>* value2 = new_node(24);
64      Node<int>* value3 = new_node(36);
65      Node<int>* value4 = new_node(48);
66      Node<int>* value5 = new_node(60);
67
68      value1->next = value2;
69      value2->next = value3;
70      value3->next = value4;
71      value4->next = value5;
72      value5->next = nullptr;
73
74      std::cout << "Binary Search for Linked List:" << std::endl;
75      std::cout << "---------------------------------" << std::endl;
76      binarySearchLL(value1, 36);
77      binarySearchLL(value1, 12);
78      binarySearchLL(value1, 30);
79      std::cout << std::endl;
80
81  }
```

Output

```
Binary Search for Linked List:
---------------------------------
Searching is successful.
Searching is successful.
Searching is unsuccessful.
```

Observations

The binary search for the linked list was successfully implemented using the ***getMiddle()*** and ***binarySearchLL()*** functions. The important code ***Node<T>* slow = start*** and ***Node<T>* fast = start*** was used to find the middle node, where ***slow*** moves one step while ***fast*** moves two steps. The ***while (fast != last && fast->next != last)*** loop continues until the middle node is reached, and ***return slow*** gives the address of that node. In the ***binarySearchLL()*** function, ***Node<T>* last = nullptr*** sets the end of the search range, while ***middle = getMiddle(start, last)*** finds the middle node for checking. The ***if (middle->data == dataFind)*** checks if the searched

	value is found, while ***last = middle*** and ***start = middle->next*** are important for moving the search to the correct half. In the main file, the values ***12, 24, 36, 48, 60*** were connected in sorted order, which is needed for binary search. The three searches correctly showed ***successful*** results for ***36*** and ***12***, while ***30*** showed an ***unsuccessful*** result because it is not in the linked list. Finally, the code worked correctly and helped show how binary search can be done in a linked list by finding the middle node and reducing the search area each time.

Table 6-3b. Binary Search for Linked List

My Output: (Complete)

```
93 14 12 81 62 80 76 5 21 83 65 25 80 6 94 77 43 21 5 52 99 75 12 34 34 83
26 6 24 9 60 22 82 32 24 51 45 92 40 21 21 34 81 15 75 9 2 39 56 8

Linear Search for Arrays:
---------------------------------
What number to search? 77
Searching is successful.

Linear Search for Linked List:
---------------------------------
Searching is successful.
Searching is unsuccessful.

Binary Search for Arrays:
---------------------------------
Searching is successful.
Searching is unsuccessful.

Binary Search for Linked List:
---------------------------------
Searching is successful.
Searching is successful.
Searching is unsuccessful.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

My Work:

Sequential Search using Array

My Code: (searchingAlgo_ILOB.h)

ILOB > C searchingAlgo_ILOB.h > ...

```
#ifndef SEARCHINGALGO_ILOB_H
#define SEARCHINGALGO_ILOB_H

#include <iostream>

// Sequential Search using Array
template <typename T>
int linearSearchArrCount(int arrSize, T dataFind, T list[]) {

    int comparisons = 0;

    // Search each element of the array
    for (int i = 0; i < arrSize; i++) {

        comparisons++;

        // Check if the current element matches the key
        if (dataFind == list[i]) {
            return comparisons;
        }
    }

    return comparisons;
}
```

My Code: (main_ILOB.cpp)

```cpp
#include <iostream>
#include "node_ILOB.h"
#include "searchingAlgo_ILOB.h"

int main() {

    // PROBLEM 1 AND 2 GIVEN LIST
    int numbers[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int arrSize = 10;
    int dataFind = 18;

    // PROBLEM 1
    std::cout << "\nPROBLEM 1:" << std::endl;
    std::cout << "----------------------------------------" << std::endl;

    // Sequential Search using Array
    std::cout << "Sequential Search using Array:" << std::endl;
    std::cout << "Key to find: " << dataFind << std::endl;

    int arrComparisons = linearSearchArrCount(
        arrSize, dataFind, numbers
    );

    std::cout << "Number of comparisons: " << arrComparisons << std::endl;

```

My Output:

```
PROBLEM 1:
----------------------------------------
Sequential Search using Array:
Key to find: 18
Number of comparisons: 2
```

Sequential Search using Linked List	**My Code: (searchingAlgo_ILOB.h)** **My Code: (main_ILOB.cpp)**

```cpp
// Sequential Search using Linked List
template <typename T>
int linearSearchLLCount(Node<T>* head, T dataFind) {

    int comparisons = 0;

    // Continue searching until the end of the linked list
    while (head != nullptr) {

        comparisons++;

        // Check if the current node matches the key
        if (head->data == dataFind) {
            return comparisons;
        }

        // Move to the next node
        head = head->next;
    }

    return comparisons;
}
```

```cpp
// Sequential Search using Linked List
Node<int>* node1 = new_node(15);
Node<int>* node2 = new_node(18);
Node<int>* node3 = new_node(2);
Node<int>* node4 = new_node(19);
Node<int>* node5 = new_node(18);
Node<int>* node6 = new_node(0);
Node<int>* node7 = new_node(8);
Node<int>* node8 = new_node(14);
Node<int>* node9 = new_node(19);
Node<int>* node10 = new_node(14);

node1->next = node2;
node2->next = node3;
node3->next = node4;
node4->next = node5;
node5->next = node6;
```

```
43      node6->next = node7;
44      node7->next = node8;
45      node8->next = node9;
46      node9->next = node10;
47      node10->next = nullptr;
48
49      std::cout << std::endl;
50      std::cout << "Sequential Search using Linked List:" << std::endl;
51      std::cout << "Key to find: " << dataFind << std::endl;
52
53      int listComparisons = linearSearchLLCount(
54          node1, dataFind
55      );
56
57      std::cout << "Number of comparisons: " << listComparisons << std::endl;
58      std::cout << "----------------------------------------\n" << std::endl;
59
```

My Output:

```
Sequential Search using Linked List:
Key to find: 18
Number of comparisons: 2
----------------------------------------
```

Answer	The sequential search was used on both the array and linked list to find the key ***18*** from the given list ***[15, 18, 2, 19, 18, 0, 8, 14, 19, 14]***. The search starts from the first value, which is ***15***, so the first comparison is made and it does not match ***18***. The next value is ***18***, which matches the key, making the second comparison successful. In the array approach, the ***linearSearchArrCount()*** function uses ***comparisons++*** to count each value checked before the ***if (dataFind == list[i])*** statement finds the key. In the linked list approach, the ***linearSearchLLCount()*** function also uses ***comparisons++*** before checking ***head->data == dataFind***, so it also takes 2 comparisons. Therefore, both the array and linked list approaches need ***2 comparisons*** to find the first occurrence of ***18***.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

My Work:

Count Repeating Instances using Array

My Code: (searchingAlgo_ILOB.h)

```
49  // Count Repeating Instances using Array
50  template <typename T>
51  int countRepeatingArr(int arrSize, T dataFind, T list[]) {
52
53      int count = 0;
54
55      // Search each element of the array
56      for (int i = 0; i < arrSize; i++) {
57
58          // Check if the current element matches the key
59          if (dataFind == list[i]) {
60              count++;
61          }
62      }
63
64      return count;
65  }
```

My Code: (main_ILOB.cpp)

```
60      // PROBLEM 2
61      std::cout << "PROBLEM 2:" << std::endl;
62      std::cout << "-------------------------------------" << std::endl;
63
64      // Count repeating instances using Array
65      std::cout << "Repeating Instances using Array:" << std::endl;
66      std::cout << "Key to find: " << dataFind << std::endl;
67
68      int arrCount = countRepeatingArr(
69          arrSize, dataFind, numbers
70      );
71
72      std::cout << "Number of repeating instances: " << arrCount << std::endl;
73
```

	My Output:
	
	`PROBLEM 2:`
	`----------`
	`Repeating Instances using Array:`
	`Key to find: 18`
	`Number of repeating instances: 2`

Count Repeating Instances using Linked List	**My Code: (searchingAlgo_ILOB.h)**
	
	``
	// Count Repeating Instances using Linked List
	template <typename T>
	int countRepeatingLL(Node<T>* head, T dataFind) {
	
	    int count = 0;
	
	    // Continue searching until the end of the linked list
	    while (head != nullptr) {
	
	        // Check if the current node matches the key
	        if (head->data == dataFind) {
	            count++;
	        }
	
	        // Move to the next node
	        head = head->next;
	    }
	
	    return count;
	}
	``

My Code: (main_ILOB.cpp)

```
74      // Count repeating instances using Linked List
75      std::cout << std::endl;
76      std::cout << "Repeating Instances using Linked List:" << std::endl;
77      std::cout << "Key to find: " << dataFind << std::endl;
78
79      int listCount = countRepeatingLL(
80          node1, dataFind
81      );
82
83      std::cout << "Number of repeating instances: " << listCount << std::endl;
84      std::cout << "--------------------------------------\n" << std::endl;
85
```

My Output:

```
Repeating Instances using Linked List:
Key to find: 18
Number of repeating instances: 2
----------------------------------------
```

Answer

The sequential search algorithm was modified so it can count all the repeating instances of the given key ***18*** in the list ***[15, 18, 2, 19, 18, 0, 8, 14, 19, 14]***. In the array approach, the ***countRepeatingArr()*** function checks every element using the ***for*** loop, and the important ***if (dataFind == list[i])*** statement checks if the current value is equal to **18**. Whenever a match is found, ***count++*** increases the number of occurrences, and the search continues until all 10 elements are checked. The same process is done in the linked list using ***countRepeatingLL()***, where ***head->data == dataFind*** checks each node and ***head = head->next*** moves to the next node. The key ***18*** appears at the second and fifth positions of the list, so both the array and linked list correctly return ***2 repeating instances***. Finally, the modified code worked properly because it continues searching the whole list instead of stopping at the first match, which makes it possible to count all occurrences of the given key.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you want to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

My Work:

Binary Search using Array

My Code: (searchingAlgo_ILOB.h)

```
88  // Binary Search
89  template <typename T>
90  int binarySearchArr(int arrSize, T dataFind, T list[]) {
91
92      // Define low, high, and middle
93      int low = 0;
94      int up = arrSize - 1;
95      int comparisons = 0;
96
97      // Loop while low is less than or equal to high
98      while (low <= up) {
99
100         // Compute for middle
101         int middle = (low + up) / 2;
102
103         comparisons++;
104
105         // If data found is middle -> data found -> return
106         if (dataFind == list[middle]) {
107             std::cout << "Key " << dataFind
108                       << " was found at index " << middle << std::endl;
```

```
109             return comparisons;
110         }
111
112         // If data find < data middle -> new high
113         else if (dataFind < list[middle]) {
114             up = middle - 1;
115         }
116
117         else {
118
119             // Else update low = middle + 1
120             low = middle + 1;
121         }
122     }
123
```

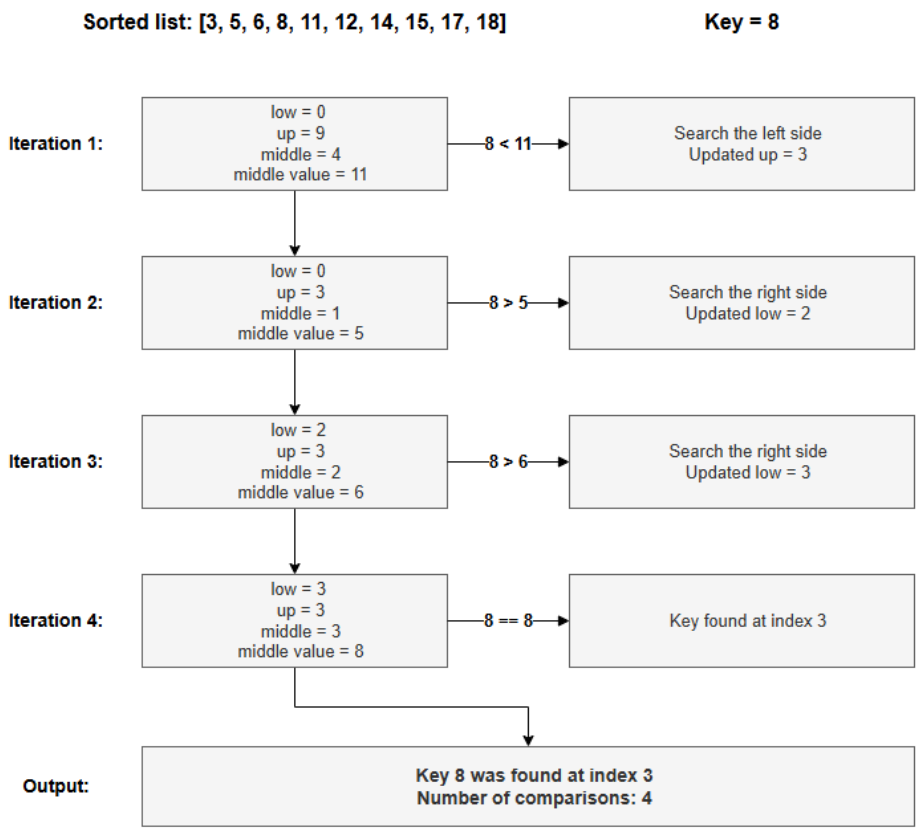
```
124     // Search is not found
125     std::cout << "Key was not found." << std::endl;
126     return comparisons;
127 }
```

My Code: (main_ILOB.cpp)

```
86      // PROBLEM 3
87      int binaryNumbers[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
88      int binarySize = 10;
89      int binaryKey = 8;
90
91      std::cout << "PROBLEM 3:" << std::endl;
92      std::cout << "---------------------------------------" << std::endl;
93
94      // Binary Search using Array
95      std::cout << "Binary Search using Array:" << std::endl;
96      std::cout << "Key to find: " << binaryKey << std::endl;
97
98      int binaryComparisons = binarySearchArr(
99          binarySize, binaryKey, binaryNumbers
100     );
101
102     std::cout << "Number of comparisons: " << binaryComparisons << std::endl;
103     std::cout << "---------------------------------------\n" << std::endl;
104
```

My Output:

```
PROBLEM 3:
---------------------------------------
Binary Search using Array:
Key to find: 8
Key 8 was found at index 3
Number of comparisons: 4
---------------------------------------
```

Diagram	
Answer	The binary search algorithm was used to find the key ***8*** from the sorted list ***[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]***. In the first iteration, the code used ***low = 0*** and ***up = 9***, making ***middle = 4***, where the middle value was ***11***. Since ***8 < 11***, the search moved to the left side by updating ***up = middle - 1***, making ***up = 3***. In the second iteration, ***middle = 1*** and the middle value was ***5***, so because ***8 > 5***, the search moved to the right side using ***low = middle + 1***, making ***low = 2***. In the third iteration, the middle value became ***6***, so the search again moved right and updated ***low = 3***. In the fourth iteration, ***middle = 3*** and the value was ***8***, which matched the key, so the search was successful. The important code ***comparisons++*** counted each middle value checked, giving a total of ***4 comparisons***, and the output also showed that key 8 was found at ***index 3***. Finally, the diagram matched the code correctly and showed how the search area became smaller in every iteration until the key was found.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

My Work:

Recursive Binary Search using Array

My Code: (searchingAlgo_ILOB.h)

```
129  // Recursive Binary Search
130  template <typename T>
131  int recursiveBinarySearch(
132      T list[], int low, int up, T dataFind, int& comparisons
133  ) {
134
135      // Check if the search range is no longer valid
136      if (low > up) {
137          return -1;
138      }
139
140      // Compute for middle
141      int middle = (low + up) / 2;
142
143      comparisons++;
144
145      // If data found is middle -> data found -> return
146      if (list[middle] == dataFind) {
147          return middle;
148      }
149
150      // If data find < data middle -> search the left side
151      else if (dataFind < list[middle]) {
152          return recursiveBinarySearch(
153              list, low, middle - 1, dataFind, comparisons
154          );
155      }
156
157      else {
158
159          // Else search the right side
160          return recursiveBinarySearch(
161              list, middle + 1, up, dataFind, comparisons
162          );
163      }
164  }
```

My Code: (main_ILOB.cpp)

```cpp
    // PROBLEM 4
    std::cout << "PROBLEM 4:" << std::endl;
    std::cout << "---------------------------------------" << std::endl;

    // Recursive Binary Search using Array
    std::cout << "Recursive Binary Search using Array:" << std::endl;
    std::cout << "Key to find: " << binaryKey << std::endl;

    int recursiveComparisons = 0;

    int recursiveResult = recursiveBinarySearch(
        binaryNumbers, 0, binarySize - 1, binaryKey, recursiveComparisons
    );

    if (recursiveResult != -1) {
        std::cout << "Key " << binaryKey << " was found at index " << recursiveResult << std::endl;
    }
    else {
        std::cout << "Key was not found." << std::endl;
    }

    std::cout << "Number of comparisons: " << recursiveComparisons << std::endl;
    std::cout << "---------------------------------------\n" << std::endl;

    return 0;
}
```

My Output:

```
PROBLEM 4:
---------------------------------------
Recursive Binary Search using Array:
Key to find: 8
Key 8 was found at index 3
Number of comparisons: 4
---------------------------------------
```

Answer	The binary search algorithm was modified into a recursive version to solve the same search problem from Problem 3. The important part of the code is the ***recursiveBinarySearch()*** function, which calls itself again when the key is not found in the current middle position. The ***if (low > up)*** condition works as the stopping point of the recursion when there are no more values to search. The code then calculates the middle position using ***int middle = (low + up) / 2*** and increases the comparison count using ***comparisons++***. If the key is smaller than the middle value, the function searches the left side using ***middle - 1***, while if the key is larger, it searches the right side using ***middle + 1***. For the sorted list ***[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]***, the key ***8*** was checked against ***11, 5, 6,*** and finally ***8***, so it was found at ***index 3*** after ***4 comparisons***. Finally, the recursive version worked correctly and showed how the same binary search process can be repeated through function calls until the key is found.

My Output: (Complete)

Part 1

```
PROBLEM 1:
--------------------------------------
Sequential Search using Array:
Key to find: 18
Number of comparisons: 2

Sequential Search using Linked List:
Key to find: 18
Number of comparisons: 2
--------------------------------------

PROBLEM 2:
--------------------------------------
Repeating Instances using Array:
Key to find: 18
Number of repeating instances: 2
```

Part 2

```
Repeating Instances using Linked List:
Key to find: 18
Number of repeating instances: 2
--------------------------------------

PROBLEM 3:
--------------------------------------
Binary Search using Array:
Key to find: 8
Key 8 was found at index 3
Number of comparisons: 4
--------------------------------------

PROBLEM 4:
--------------------------------------
Recursive Binary Search using Array:
Key to find: 8
Key 8 was found at index 3
Number of comparisons: 4
--------------------------------------
```

Conclusion:

The activity helped in learning how searching techniques work and how they can be applied using C++. For the lessons learned, sequential search checks the data one by one, while binary search checks the middle value and reduces the search area, which makes the process more efficient when the data is sorted. The activity also showed how the same searching idea can be used in both arrays and linked lists, and how a binary search can be changed into a recursive function. For the procedure, the codes were created and tested step by step, starting with generating and storing data, then applying sequential search and binary search. The important parts such as linearSearchArr(), linearSearchLL(), binarySearchArr(), getMiddle(), and binarySearchLL() helped show how each searching method works. For the supplementary activity, the search algorithms were modified to count comparisons, count repeating instances, and use recursive binary search.

One difficulty faced was understanding how linked lists move through the next pointer and how the middle node is found during binary search. Another difficulty was keeping track of low, up, and middle during each binary search iteration, especially when making the recursive version. Even with these difficulties, the results were correct and the output matched the expected answers. Overall, the activity was done well because the searching techniques were understood and successfully implemented, but there is still an area for improvement in understanding linked list operations, recursion, and making the code more familiar without depending too much on examples. More practice with different searching problems can help improve the understanding and confidence in using these algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.